eddelbuettel / pinp

Code Issues 3 Pull requests Actions Projects Security

Dismiss

Join GitHub today

GitHub is home to over 50 million developers working together to host and review code, manage projects, and build software together.

Sign up

Branch: master pinp / README.md Go to file ...

eddelbuettel added 'last commit' bad... Latest commit 165f79c on Apr 9 History

2 contributors

125 lines (85 sloc) 4.37 KB Raw Blame

pinp: Pinp is not PNAS

build passing | license GPL-3 | CRAN 0.0.9 | dependencies 2/18 | downloads 2086/month | last commit april

Motivation

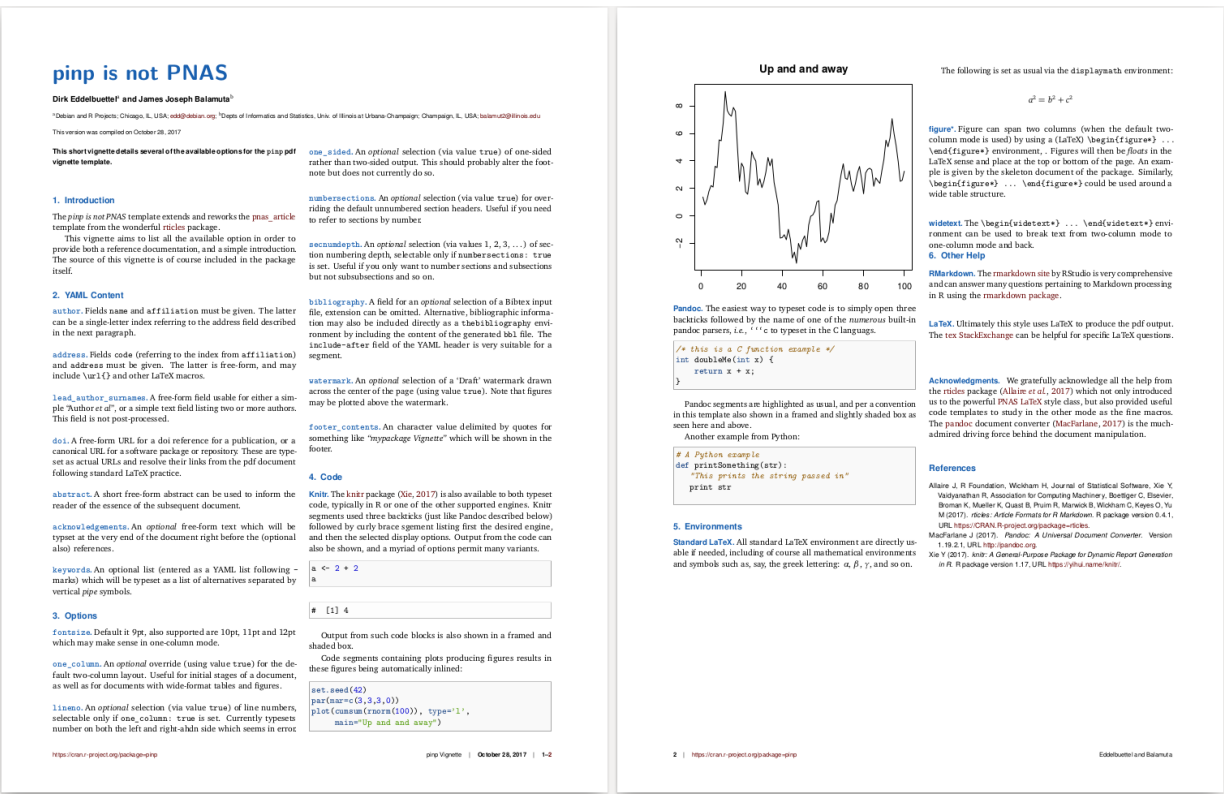
The template provided by the PNAS Article template in the rticles package makes it very easy and convenient to create attractive looking two-column papers with an option to switch to a one-column paper.

There were however a few changes we wanted to make such as the use of standard natbib citations, more extended use of color, a different font as well as the removal of a number of PNAS-specific title page items as well as general code and use simplification via a single LaTeX class file. So this *pinp* package started.

Example

Pinp Vignette

As of release 0.0.2, the package now has a vignette:



pinp is not PNAS

Dirk Eddelbuettel[a] and James Joseph Balamuta[b]

[a]Debian and R Projects; Chicago, IL, USA; edd@debian.org; [b]Depts of Informatics and Statistics, Univ. of Illinois at Urbana-Champaign; Champaign, IL, USA; balamut2@illinois.edu

This version was compiled on October 28, 2017

This short vignette details several of the available options for the `pinp` pdf vignette template.

1. Introduction

The *pinp is not PNAS* template extends and reworks the pnas_article template from the wonderful rticles package.

This vignette aims to list all the available option in order to provide both a reference documentation, and a simple introduction. The source of this vignette is of course included in the package itself.

2. YAML Content

`author`. Fields `name` and `affiliation` must be given. The latter can be a single-letter index referring to the address field described in the next paragraph.

`address`. Fields `code` (referring to the index from `affiliation`) and `address` must be given. The latter is free-form, and may include `\url{}` and other LaTeX macros.

`lead_author_surnames`. A free-form field usable for either a simple "Author *et al*", or a simple text field listing two or more authors. This field is not post-processed.

`doi`. A free-form URL for a doi reference for a publication, or a canonical URL for a software package or repository. These are typeset as actual URLs and resolve their links from the pdf document following standard LaTeX practice.

`abstract`. A short free-form abstract can be used to inform the reader of the essence of the subsequent document.

`acknowledgements`. An *optional* free-form text which will be typset at the very end of the document right before the (optional also) references.

`keywords`. An optional list (entered as a YAML list following `-` marks) which will be typeset as a list of alternatives separated by vertical *pipe* symbols.

3. Options

`fontsize`. Default it `9pt`, also supported are `10pt`, `11pt` and `12pt` which may make sense in one-column mode.

`one_column`. An *optional* override (using value `true`) for the default two-column layout. Useful for initial stages of a document, as well as for documents with wide-format tables and figures.

`lineno`. An *optional* selection (via value `true`) of line numbers, selectable only if `one_column: true` is set. Currently typesets number on both the left and right-ahdn side which seems in error

`one_sided`. An *optional* selection (via value `true`) of one-sided rather than two-sided output. This should probably alter the footnote but does not currently do so.

`numbersections`. An *optional* selection (via value `true`) for overriding the default unnumbered section headers. Useful if you need to refer to sections by number.

`secnumdepth`. An *optional* selection (via values 1, 2, 3, ...) of section numbering depth, selectable only if `numbersections: true` is set. Useful if you only want to number sections and subsections but not subsubsections and so on.

`bibliography`. A field for an *optional* selection of a Bibtex input file, extension can be omitted. Alternative, bibliographic information may also be included directly as a `thebibliography` environment by including the content of the generated `bbl` file. The `include-after` field of the YAML header is very suitable for a segment.

`watermark`. An *optional* selection of a 'Draft' watermark drawn across the center of the page (using value `true`). Note that figures may be plotted above the watermark.

`footer_contents`. An character value delimited by quotes for something like *"mypackage Vignette"* which will be shown in the footer.

4. Code

Knitr. The knitr package (Xie, 2017) is also available to both typeset code, typically in R or one of the other supported engines. Knitr segments used three backticks (just like Pandoc described below) followed by curly brace segment listing first the desired engine, and then the selected display options. Output from the code can also be shown, and a myriad of options permit many variants.

```
a <- 2 + 2
a
```

```
#  [1] 4
```

Output from such code blocks is also shown in a framed and shaded box.

Code segments containing plots producing figures results in these figures being automatically inlined:

```
set.seed(42)
par(mar=c(3,3,3,0))
plot(cumsum(rnorm(100)), type='l',
     main="Up and away")
```

Up and away

The following is set as usual via the `displaymath` environment:

$$a^2 = b^2 + c^2$$

`figure*`. Figure can span two columns (when the default two-column mode is used) by using a (LaTeX) `\begin{figure*} ... \end{figure*}` environment, . Figures will then be *floats* in the LaTeX sense and place at the top or bottom of the page. An example is given by the skeleton document of the package. Similarly, `\begin{figure*} ... \end{figure*}` could be used around a wide table structure.

`widetext`. The `\begin{widetext*} ... \end{widetext*}` environment can be used to break text from two-column mode to one-column mode and back.

6. Other Help

RMarkdown. The rmarkdown site by RStudio is very comprehensive and can answer many questions pertaining to Markdown processing in R using the rmarkdown package.

LaTeX. Ultimately this style uses LaTeX to produce the pdf output. The tex StackExchange can be helpful for specific LaTeX questions.

Pandoc. The easiest way to typeset code is to simply open three backticks followed by the name of one of the *numerous* built-in pandoc parsers, *i.e.*, `'''c` to typeset in the C languages.

```
/* this is a C function example */
int doubleMe(int x) {
    return x + x;
}
```

Pandoc segments are highlighted as usual, and per a convention in this template also shown in a framed and slightly shaded box as seen here and above.

Another example from Python:

```
# A Python example
def printSomething(str):
    "This prints the string passed in"
    print str
```

5. Environments

Standard LaTeX. All standard LaTeX environment are directly usable if needed, including of course all mathematical environments and symbols such as, say, the greek lettering: α, β, γ, and so on.

Acknowledgments. We gratefully acknowledge all the help from the rticles package (Allaire *et al.*, 2017) which not only introduced us to the powerful PNAS LaTeX style class, but also provided useful code templates to study in the other mode as the fine macros. The pandoc document converter (MacFarlane, 2017) is the much-admired driving force behind the document manipulation.

References

Allaire J, R Foundation, Wickham H, Journal of Statistical Software, Xie Y, Vaidyanathan R, Association for Computing Machinery, Boettiger C, Elsevier, Broman K, Mueller K, Quast B, Pruim R, Marwick B, Wickham C, Keyes O, Yu M (2017). *rticles: Article Formats for R Markdown*. R package version 0.4.1, URL https://CRAN.R-project.org/package=rticles.

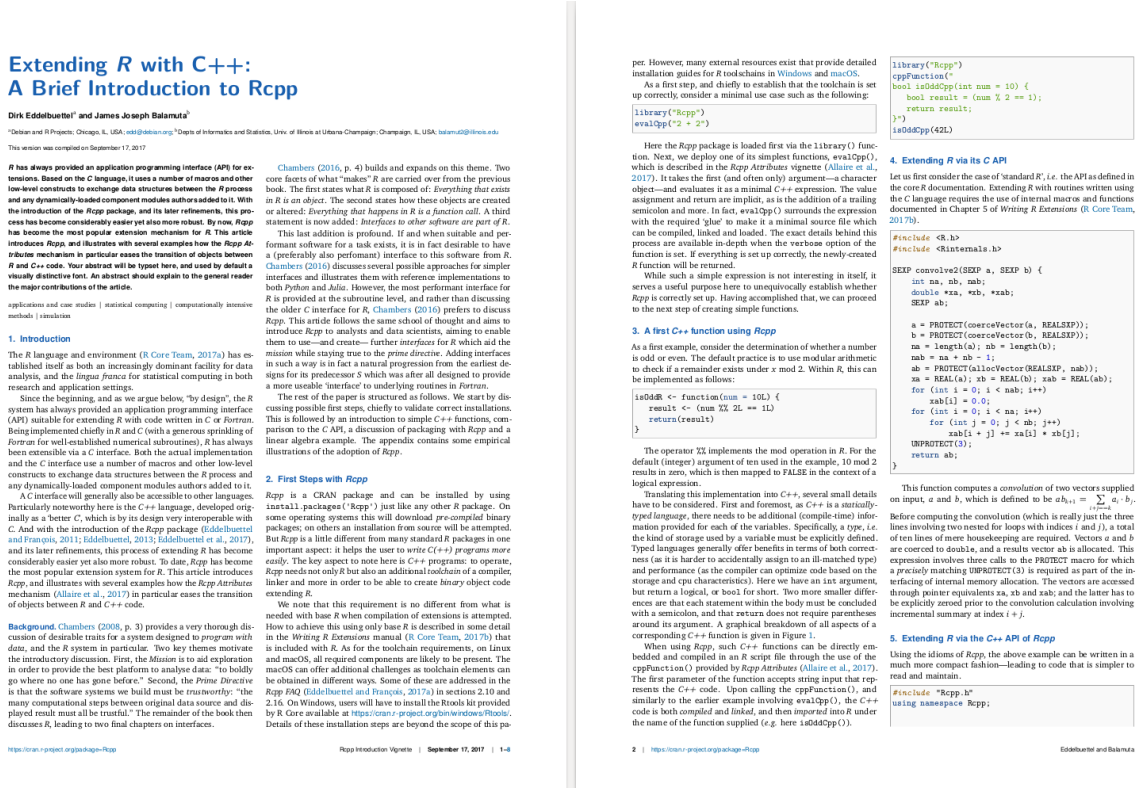
MacFarlane J (2017). *Pandoc: A Universal Document Converter*. Version 1.19.2.1, URL http://pandoc.org.

Xie Y (2017). *knitr: A General-Purpose Package for Dynamic Report Generation in R*. R package version 1.17, URL https://yihui.name/knitr/.

Rcpp Vignette

A complete pdf example of the two-column default template is provided here by the new introductory Rcpp vignette, which is itself based on our PeerJ Prepint.

A screenshot of the first two pages:



Extending *R* with C++: A Brief Introduction to Rcpp

Dirk Eddelbuettel[a] and James Joseph Balamuta[b]

[a]Debian and R Projects; Chicago, IL, USA; edd@debian.org [b]Depts of Informatics and Statistics, Univ. of Illinois at Urbana-Champaign; Champaign, IL, USA; balamut2@illinois.edu

This version was compiled on September 17, 2017

***R* has always provided an application programming interface (API) for extensions. Based on the *C* language, it uses a number of macros and other low-level constructs to exchange data structures between the *R* process and any dynamically-loaded component modules authors added to it. With the introduction of the *Rcpp* package, and its later refinements, this process has become considerably easier yet also more robust. By now, *Rcpp* has become the most popular extension mechanism for *R*. This article introduces *Rcpp*, and illustrates with several examples how the *Rcpp Attributes* mechanism in particular eases the transition of objects between *R* and *C++* code. Your abstract will be typset here, and used by default a visually distinctive font. An abstract should explain to the general reader the major contributions of the article.**

applications and case studies | statistical computing | computationally intensive methods | simulation

1. Introduction

The *R* language and environment (R Core Team, 2017a) has established itself as both an increasingly dominant facility for data analysis, and the *lingua franca* for statistical computing in both research and application settings.

Since the beginning, and as we argue below, "by design", the *R* system has always provided an application programming interface (API) suitable for extending *R* with code written in *C* or *Fortran*. Being implemented chiefly in *R* and *C* (with a generous sprinkling of *Fortran* for well-established numerical subroutines), *R* has always been extensible via a *C* interface. Both the actual implementation and the *C* interface use a number of macros and other low-level constructs to exchange data structures between the *R* process and any dynamically-loaded component modules authors added to it.

A *C* interface will generally also be acessible to other languages. Particularly noteworthy here is the *C++* language, developed originally as a 'better *C*', which is by its design very interoperable with *C*. And with the introduction of the *Rcpp* package (Eddelbuettel and François, 2011; Eddelbuettel, 2013; Eddelbuettel et al., 2017), and its later refinements, this process of extending *R* has become considerably easier yet also more robust. To date, *Rcpp* has become the most popular extension system for *R*. This article introduces *Rcpp*, and illustrates with several examples how the *Rcpp Attributes* mechanism (Allaire et al., 2017) in particular eases the transition of objects between *R* and *C++* code.

Background. Chambers (2008, p. 3) provides a very thorough discussion of desirable traits for a system designed to *program with data*, and the *R* system in particular. Two key themes motivate the introductory discussion. First, the *Mission* is to aid exploration in order to provide the best platform to analyse data: "to boldly go where no one has gone before." Second, the *Prime Directive* is that the software systems we build must be *trustworthy*: "the many computational steps between original data source and displayed result must all be trustful." The remainder of the book then discusses *R*, leading to two final chapters on interfaces.

Chambers (2016, p. 4) builds and expands on this theme. Two core facets of what "makes" *R* are carried over from the previous book. The first states what *R* is composed of: *Everything that exists in R is an object*. The second states how these objects are created or altered: *Everything that happens in R is a function call*. A third statement is now added: *Interfaces to other software are part of R*.

This last addition is profound. If and when suitable and performant software for a task exists, it is in fact desirable to have a (preferably also perfomant) interface to this software from *R*. Chambers (2016) discusses several possible approaches for simpler interfaces and illustrates them with reference implementations to both *Python* and *Julia*. However, the most performant interface for *R* is provided at the subroutine level, and rather than discussing the older *C* interface for *R*, Chambers (2016) prefers to discuss *Rcpp*. This article follows the same school of thought and aims to introduce *Rcpp* to analysts and data scientists, aiming to enable them to use—and create— further *interfaces* for *R* which aid the *mission* while staying true to the *prime directive*. Adding interfaces in such a way is in fact a natural progression from the earliest designs for its predecessor *S* which was after all designed to provide a more useable 'interface' to underlying routines in *Fortran*.

The rest of the paper is structured as follows. We start by discussing possible first steps, chiefly to validate correct installations. This is followed by an introduction to simple *C++* functions, comparison to the *C* API, a discussion of packaging with *Rcpp* and a linear algebra example. The appendix contains some empirical illustrations of the adoption of *Rcpp*.

2. First Steps with *Rcpp*

Rcpp is a CRAN package and can be installed by using `install.packages('Rcpp')` just like any other *R* package. On some operating systems this will download *pre-compiled* binary packages; on others an installation from source will be attempted. But *Rcpp* is a little different from many standard *R* packages in one important aspect: it helps the user to *write C(++) programs more easily*. The key aspect to note here is *C++* programs: to operate, *Rcpp* needs not only *R* but also an additional *toolchain* of a compiler, linker and more in order to be able to create *binary* object code extending *R*.

We note that this requirement is no different from what is needed with base *R* when compilation of extensions is attempted. How to achieve this using only base *R* is described in some detail in the *Writing R Extensions* manual (R Core Team, 2017b) that is included with *R*. As for the toolchain requirements, on Linux and macOS, all required components are likely to be present. The macOS can offer additional challenges as toolchain elements can be obtained in different ways. Some of these are addressed in the *Rcpp FAQ* (Eddelbuettel and François, 2017a) in sections 2.10 and 2.16. On Windows, users will have to install the Rtools kit provided by R Core available at https://cran.r-project.org/bin/windows/Rtools/. Details of these installation steps are beyond the scope of this paper. However, many external resources exist that provide detailed installation guides for *R* toolchains in Windows and macOS.

As a first step, and chiefly to establish that the toolchain is set up correctly, consider a minimal use case such as the following:

```
library("Rcpp")
evalCpp("2 + 2")
```

Here the *Rcpp* package is loaded first via the `library()` function. Next, we deploy one of its simplest functions, `evalCpp()`, which is described in the *Rcpp Attributes* vignette (Allaire et al., 2017). It takes the first (and often only) argument—a character object—and evaluates it as a minimal *C++* expression. The value assignment and return are implicit, as is the addition of a trailing semicolon and more. In fact, `evalCpp()` surrounds the expression with the required 'glue' to make it a minimal source file which can be compiled, linked and loaded. The exact details behind this process are available in-depth when the `verbose` option of the function is set. If everything is set up correctly, the newly-created *R* function will be returned.

While such a simple expression is not interesting in itself, it serves a useful purpose here to unequivocally establish whether *Rcpp* is correctly set up. Having accomplished that, we can proceed to the next step of creating simple functions.

3. A first *C++* function using *Rcpp*

As a first example, consider the determination of whether a number is odd or even. The default practice is to use modular arithmetic to check if a remainder exists under x mod 2. Within *R*, this can be implemented as follows:

```
isOddR <- function(num = 10L) {
  result <- (num %% 2L == 1L)
  return(result)
}
```

The operator `%%` implements the mod operation in *R*. For the default (integer) argument of ten used in the example, 10 mod 2 results in zero, which is then mapped to `FALSE` in the context of a logical expression.

Translating this implementation into *C++*, several small details have to be considered. First and foremost, as *C++* is a *statically-typed language*, there needs to be additional (compile-time) information provided for each of the variables. Specifically, a *type*, *i.e.* the kind of storage used by a variable must be explicitly defined. Typed languages generally offer benefits in terms of both correctness (as it is harder to accidentally assign to an ill-matched type) and performance (as the compiler can optimize code based on the storage and cpu characteristics). Here we have an `int` argument, but return a logical, or `bool` for short. Two more smaller differences are that each statement within the body must be concluded with a semicolon, and that `return` does not require parentheses around its argument. A graphical breakdown of all aspects of a corresponding *C++* function is given in Figure 1.

When using *Rcpp*, such *C++* functions can be directly embedded and compiled in an *R* script file through the use of the `cppFunction()` provided by *Rcpp Attributes* (Allaire et al., 2017). The first parameter of the function accepts string input that represents the *C++* code. Upon calling the `cppFunction()`, and similarly to the earlier example involving `evalCpp()`, the *C++* code is both *compiled* and *linked*, and then *imported* into *R* under the name of the function supplied (*e.g.* here `isOddCpp()`).

```
library("Rcpp")
cppFunction("
bool isOddCpp(int num = 10) {
  bool result = (num % 2 == 1);
  return result;
}")
isOddCpp(42L)
```

4. Extending *R* via its *C* API

Let us first consider the case of 'standard *R*', *i.e.* the API as defined in the core *R* documentation. Extending *R* with routines written using the *C* language requires the use of internal macros and functions documented in Chapter 5 of *Writing R Extensions* (R Core Team, 2017b).

```
#include <R.h>
#include <Rinternals.h>

SEXP convolve2(SEXP a, SEXP b) {
    int na, nb, nab;
    double *xa, *xb, *xab;
    SEXP ab;

    a = PROTECT(coerceVector(a, REALSXP));
    b = PROTECT(coerceVector(b, REALSXP));
    na = length(a); nb = length(b);
    nab = na + nb - 1;
    ab = PROTECT(allocVector(REALSXP, nab));
    xa = REAL(a); xb = REAL(b); xab = REAL(ab);
    for (int i = 0; i < nab; i++)
        xab[i] = 0.0;
    for (int i = 0; i < na; i++)
        for (int j = 0; j < nb; j++)
            xab[i + j] += xa[i] * xb[j];
    UNPROTECT(3);
    return ab;
}
```

This function computes a *convolution* of two vectors supplied on input, a and b, which is defined to be $ab_{k+1} = \sum_{i+j=k} a_i \cdot b_j$. Before computing the convolution (which is really just the three lines involving two nested for loops with indices i and j), a total of ten lines of mere housekeeping are required. Vectors a and b are coerced to `double`, and a results vector `ab` is allocated. This expression involves three calls to the `PROTECT` macro for which a *precisely* matching `UNPROTECT(3)` is required as part of the interfacing of internal memory allocation. The vectors are accessed through pointer equivalents `xa`, `xb` and `xab`; and the latter has to be explicitly zeroed prior to the convolution calculation involving incremental summary at index $i + j$.

5. Extending *R* via the *C++* API of *Rcpp*

Using the idioms of *Rcpp*, the above example can be written in a much more compact fashion—leading to code that is simpler to read and maintain.

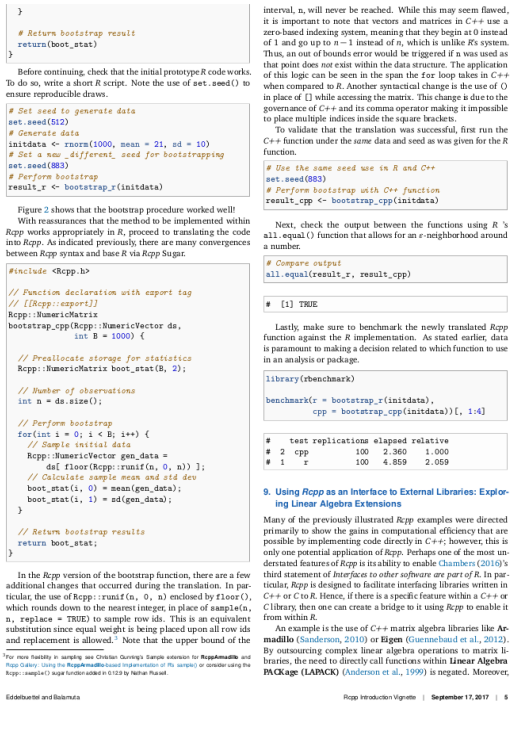
```
#include "Rcpp.h"
using namespace Rcpp;
```

https://cran.r-project.org/package=Rcpp Rcpp Introduction Vignette | September 17, 2017 | 1–8

2 | https://cran.r-project.org/package=Rcpp Eddelbuettel and Balamuta

and the next two pages, showing a two-column figure

followed by pages five and six



and the final two pages:

Other Rcpp vignettes have been converted as well.

Status

The package is working as expected, and available via CRAN.

Usage

Use the standard

```
install.packages("pinp")
```

to install it from CRAN, and then use as a Markdown template via RStudio, or use code such as

```
library(rmarkdown)
draft("mypaper.Rmd", template="pdf", package="pinp", edit=FALSE)
render("mypaper.Rmd")
```

to create a first draft of a new `mypaper.Rmd` .

Requirements

Beyond the R package dependencies, a working `pandoc` binary is needed. RStudio installs its own copy, otherwise do what is needed on your OS (*i.e.*, something like `sudo apt-get install pandoc pandoc-citeproc`).

The pdf mode requires a fairly complete LaTeX installation, and has been used on Ubuntu and macOS. On Debian/Ubuntu, the following packages should provide a working set:

```
texlive-base
texlive-binaries
texlive-fonts-extra
texlive-fonts-recommended
texlive-generic-recommended
texlive-humanities
texlive-latex-base
texlive-latex-extra
texlive-latex-recommended
texlive-pictures
texlive-publishers
texlive-science
```

See also

- binb: Binb is not Beamer: Stylish pdf Presentations from RMarkdown
- linl: Linl is not Letter: LaTeX letters from RMarkdown
- tint: tint is not tufte: Beautiful pdf or html handout or writeups

and the minm package installing all of them.

Authors

Dirk Eddelbuettel and James Balamuta, leaning on the earlier work by JJ et al in the rticles package.

The authors of the underlying PNAS LaTeX Style are not explicitedly listed, but PNAS refers to Overleaf.

License

GPL-3 for this package, as well as the short code taken with grateful acknowledgements from the rticles package.

The main PNAS LaTeX style file (and supporting files) are under the LPPL (LaTeX Project Public License) version 1.3 or later.